Merge Sort

Merge Sort is one of the most important sorting algorithms in **Design and Analysis of Algorithms**. It is based on the **Divide and Conquer** paradigm and guarantees efficient performance even for large datasets.

1️⃣ What is Merge Sort?

Merge Sort divides the array into smaller parts, sorts each part, and then merges them back together.

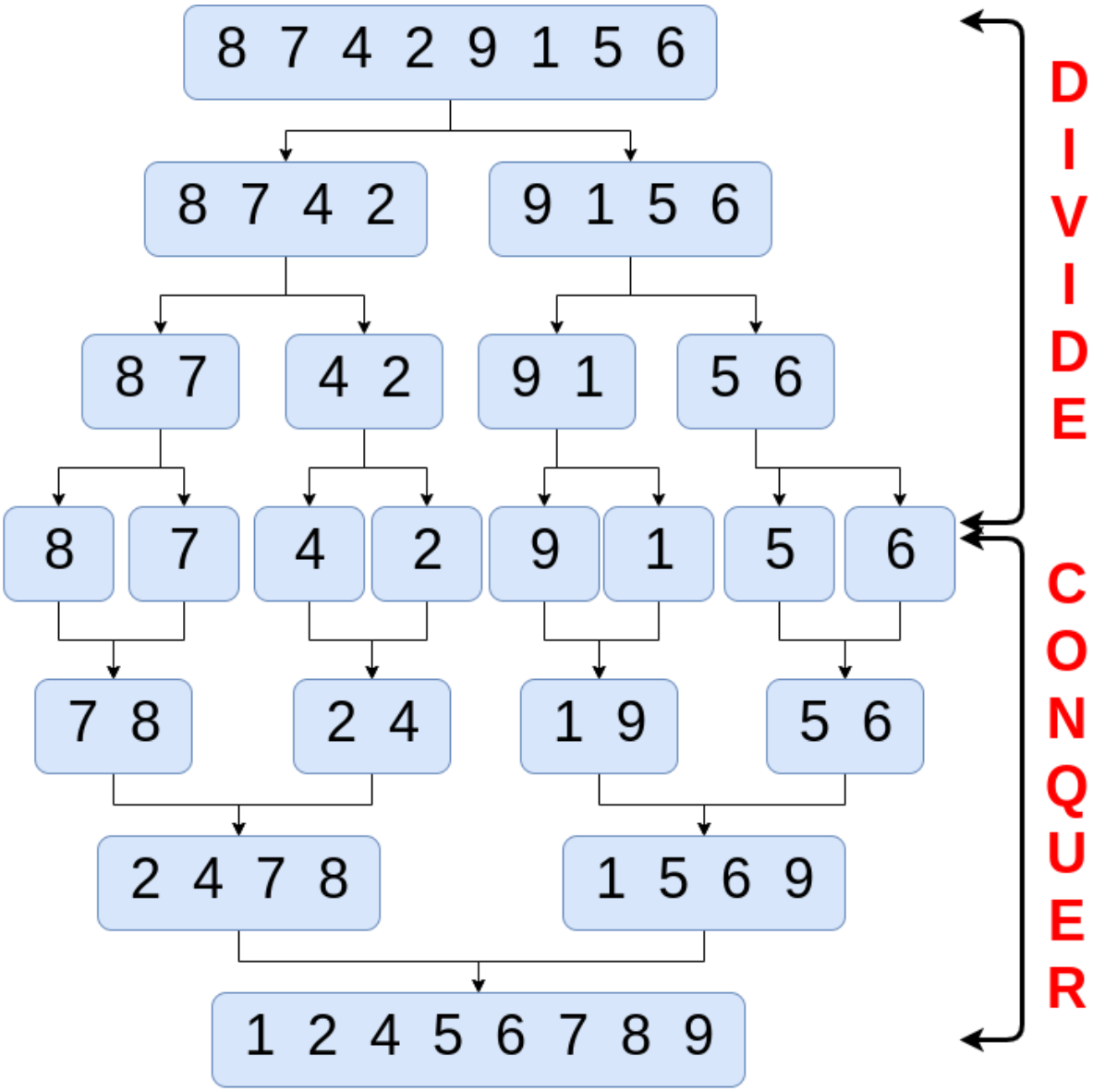
It follows three steps:

1. **Divide** → Split the array into two halves
2. **Conquer** → Sort each half recursively
3. **Combine** → Merge the sorted halves

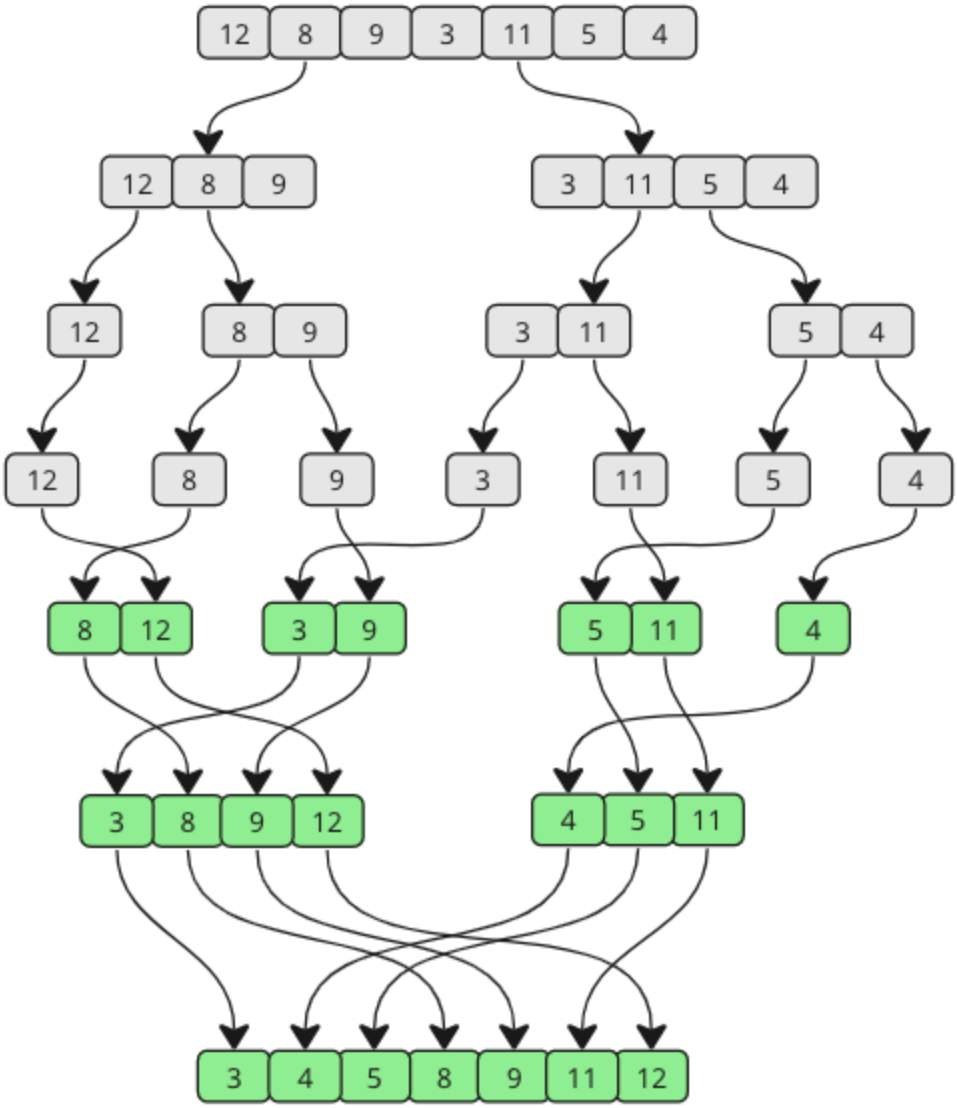
2️⃣ Divide and Conquer Concept

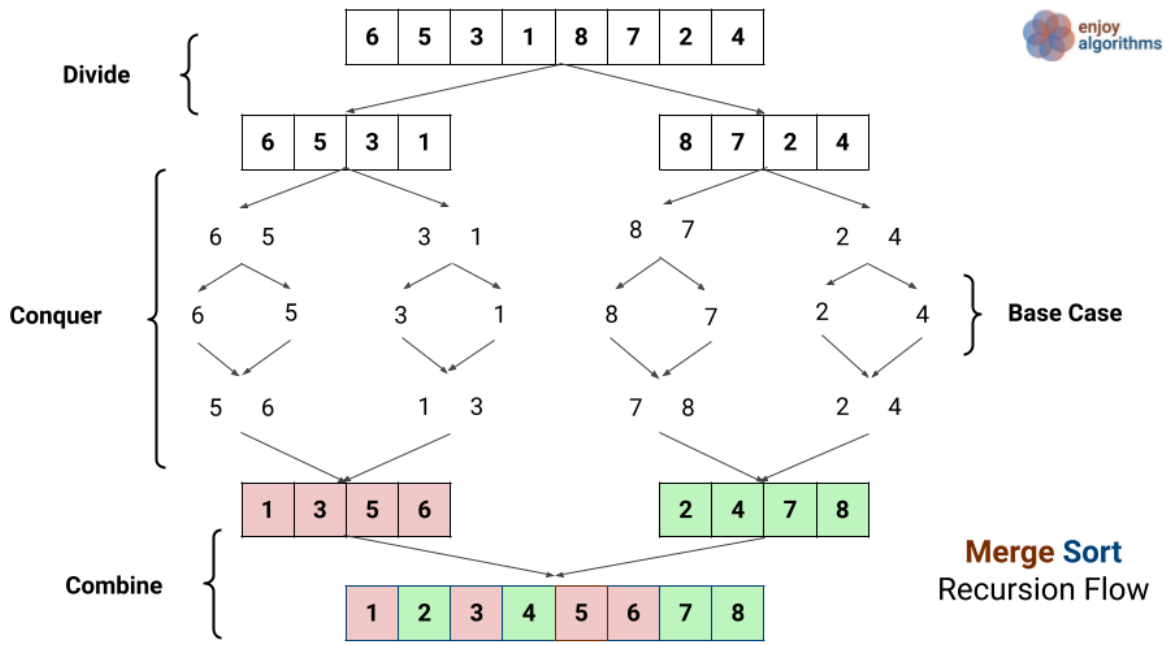
```
Divide:
[8 3 5 4 7 6 1 2]

Conquer:
[8 3 5 4]    [7 6 1 2]

Combine:
[1 2 3 4 5 6 7 8]
```

3️⃣ Visualization of Merge Sort


8 7 4 2 9 1 5 6
8 7 4 2
9 1 5 6
8 7
4 2
9 1
5 6
8
7
4
2
9
1
5
6
7 8
2 4
1 9
5 6
2 4 7 8
1 5 6 9
1 2 4 5 6 7 8 9
DIVIDE
CONQUER

Merge Sort


12 8 9 3 11 5 4
12 8 9
3 11 5 4
12
8 9
3 11
5 4
12
8
9
3
11
5
4
8 12
3 9
5 11
4
3 8 9 12
4 5 11
3 4 5 8 9 11 12



The array is repeatedly divided until single elements remain, then merged back in sorted order.

4️⃣ Basic Idea

Merge Sort keeps dividing until:

```
Single element = already sorted
```

Then merges back:

```
[3] + [8] → [3 8]
```

5️⃣ Algorithm Steps

Step 1

Divide array into two halves

Step 2

Sort left half

Step 3

Sort right half

Step 4

Merge two halves

6️⃣ Pseudocode

```
MERGE-SORT(A, left, right)
    if left < right
        mid = (left + right) / 2

        MERGE-SORT(A, left, mid)
        MERGE-SORT(A, mid+1, right)

        MERGE(A, left, mid, right)
```

7️⃣ Merge Procedure

```
MERGE(A, left, mid, right)

Create two arrays:
L = left part
R = right part

Compare elements and merge
```

8️⃣ Clean C++ Code

This version is simple and good for teaching.

```
#include <iostream>
using namespace std;

/*
    Merge two sorted subarrays:
    left part  -> arr[left ... mid]
    right part -> arr[mid+1 ... right]
*/
void merge(int arr[], int left, int mid, int right)
{
    int n1 = mid - left + 1;   // size of left subarray
    int n2 = right - mid;      // size of right subarray

    int L[n1], R[n2];

    // Copy data
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // Merge arrays
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
```

```
    // Remaining elements
    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right)
{
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int arr[] = {8, 3, 5, 4, 7, 6, 1, 2};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Before sorting: ";
    printArray(arr, n);
```

```
    mergeSort(arr, 0, n - 1);

    cout << "After sorting: ";
    printArray(arr, n);
}
```

9️⃣ Step-by-Step Example

Array:

```
8 3 5 4
```

Divide

```
8 3 | 5 4
```

Divide Again

```
8 | 3 | 5 | 4
```

Merge

```
3 8 | 4 5
```

Final Merge

```
3 4 5 8
```

🔟 Time Complexity

Merge Sort always divides the array in half.

Number of levels:

$\log n$

Work per level:

n

Total:

$O(n \log n)$

1️⃣1️⃣ Best Case

$O(n \log n)$

1️⃣2️⃣ Worst Case

$O(n \log n)$

1️⃣3️⃣ Space Complexity

Merge Sort requires extra memory:

$O(n)$

1️⃣4️⃣ Properties

Property	Value
Stable	✅ Yes
In-place	❌ No
Adaptive	❌ No
Fast	✅ Yes

1️⃣5️⃣ Why Merge Sort is Stable

Example:

```
5a 5b 3
```

After sorting:

```
3 5a 5b
```

Order preserved.

1️⃣6️⃣ Recursion Tree

```
        n
      /   \
    n/2   n/2
    / \   / \
```

Height:

```
log n
```

Total work:

```
n log n
```

1️⃣7️⃣ Advantages

✅ Guaranteed O(n log n)

✅ Stable

✅ Good for large data

✅ Works well for linked lists

1️⃣8️⃣ Disadvantages

❌ Extra memory

❌ Slower than QuickSort in practice

❌ Recursive overhead

1️⃣9️⃣ Comparison with Quick Sort

Feature	Merge	Quick
Worst Case	n log n	n^2
Space	n	log n
Stable	Yes	No
Speed	Medium	Fast

2️⃣0️⃣ Loop Invariant Idea

Before merge:

```
Left sorted
Right sorted
```

After merge:

```
Whole sorted
```

2️⃣1️⃣ Common Mistakes

❌ Wrong mid calculation

❌ Wrong subarray size

❌ Merge logic error

❌ Infinite recursion

2️⃣2️⃣ Viva Questions

1. Why is merge sort O(n log n)?
2. Why is it stable?
3. Why extra memory needed?
4. Divide and conquer?
5. Compare with quicksort.

2️⃣3️⃣ Short Exam Answer

Merge sort uses divide and conquer to sort elements.

Time complexity: O(n log n).

Space complexity: O(n).

Stable sorting algorithm.